Data-first candidate plan: 96 H100s for 30 days

Unique corpus requirements, smaller overtrained MoE examples, and bounded experiments

> **Note**
>
> **Current priority: reproduce Marin first.** The numerical MoE/loop examples below remain analytical sizing examples, not Marin architecture replicas. The Marin code/data contract, heavy tuning alternative and cyber extension define the next experiment sequence. Choose a standard reproduced baseline before adding recurrence; replace/prune the bounded queue or fund extra R&D before adopting the heavier search.

Scope and the decision we can make now

The latest allocation is **96 GPUs for 30 days**. We provisionally retain the previous H100 SXM 80GB assumption: twelve 8-GPU nodes with fast networking. Confirm GPU type, memory and topology before using these throughput scenarios. This page supersedes the 128×40 budget for current planning; that earlier guide is preserved as a historical alternative.

Prioritize a **measured data release plus trainer/evaluator qualification**, then freeze the model/horizon once measured throughput and retained bucket sizes are known. The industry audit explains why width need not be a power of two and why a hyperparameter ladder need not include a full loss-law study. The data guide gives the actual preparation stages.

The numerical examples below preserve the user's approximately 10×-Chinchilla overtraining preference, operationalized as at least 200 training tokens per stored parameter. This is a screening proxy for our MoE design, not a fitted MoE compute optimum.

Compute and two honest budget interpretations

For GPU count $G = 96$, training duration t, useful model throughput r and productive fraction $q = 0.9$,

$$C_{\text{useful}} = Gtrq, \qquad D_{\text{capacity}} = C_{\text{useful}}/f(\mathcal{A}, 4096).$$

Use model FLOPs divided by measured training wall time; recomputation, communication and optimizer overhead therefore reduce r. Do not count those overheads again by multiplying peak FLOPs into the estimate. The availability factor handles validation/checkpoint/restart downtime separately; avoid double discounting when using an end-to-end measured rate.

Interpretation	Research/preparation inside allocation	Hero days	Proposed stored size	Proposed exposures	TPP
30 days are hero training only	Separate budget, must be stated	30	14.84B	3.3T	222
30 days cover the whole GPU effort	At most 576 GPU-days = 6 whole-cluster days	24	13.64B	2.8T	205

These are **candidate dimensions to benchmark**, not selected architectures. Both use width 1536, 32 physical layers, top-8 routing, expert hidden size 1024, 24 query heads / 6 KV heads of dimension 64, untied 50,304-vocabulary embeddings, and context 4096. The 30-hero-day example has 96 experts; the all-in example has 88. The middle half is executed twice, giving 48 executed layers. A standard non-looped MoE control must be tuned and benchmarked before selecting recurrence. Counts assume per-channel QK norms and the existing analytical counter; an implementation must confirm exact tensor counts.

Analytical model FLOPs/token are about 14.89G in both cases. Expert storage falls when experts decrease, while top-k compute barely changes; this illustrates why total-parameter counts alone cannot price a MoE run. The current 20B candidate would reach only about 2.72T exposures at the nominal rate under 96×30, falling below 200 TPP, so it is not carried over unchanged.

Useful model TFLOP/s/GPU	30 hero days, 96-expert example	24 hero days, 88-expert example
150	2.26T	1.81T
200	3.01T	2.41T
250	3.76T	3.01T
300	4.51T	3.61T

The proposed 3.3T/2.8T targets leave some headroom at **250**, but cannot be promised at 200. Recompute before launch; a measured slower rate requires a smaller stored model, lower exposure target or different architecture. Changing expert count changes the statistical problem and needs a fresh check; do not delete experts mid-run to repair the budget.

Numbers are reproducible with `python scripts/industry_budget.py`; the JSON and scenario CSV retain assumptions. No GPU performance measurement has been made here.

How much unique data is needed?

There is no single answer independent of mixture and repetition. If $D_b = w_b D$ and bucket b may repeat at most $e_{b,\max}$ times, then

$$U_b \geq D_b / e_{b,\max}.$$

The following is a **testable starting mixture**, inherited from the previous proposal, not an optimal recipe from Marin. Its 80/20 phases give total weights of 23% medium web, 8% medium-low web, 14% medium-high web, 22% high web, 11% high synthetic, 16% code and 6% math. Raw low-quality web starts at zero and is included in an ablation. Upstream quality names are not Marin quality-band equivalents.

Bucket	Repetition cap	3.3T exposures: minimum unique	2.8T exposures: minimum unique
Medium actual web	1	759B	644B
Medium-low actual web	1	264B	224B
Medium-high actual web	2	231B	196B
High actual web	3	242B	205.3B
High synthetic	2	181.5B	154B
Code	4	132B	112B
Mathematics	6	33B	28B
Sum of bucket minima		**1.8425T**	**1.5633T**

These are *retained, disjoint, training-split tokens under our tokenizer*. The sum assumes bucket assignments are exclusive and global duplicate removal has happened. It does not certify semantic independence between synthetic and parent documents. Held-out data is additional. Recounting and safety margin are required; a source's headline token count cannot establish these capacities.

At no repetition, preparing all 3.3T or 2.8T unique exposures would require that many retained tokens. With the proposed caps, roughly **1.6–1.9T unique retained tokens** is a reasonable capacity objective to investigate, not a guarantee that every required high-quality/math/code pool is accessible. StarCoderData access remains gated and has not been accepted here. Resolve code sourcing before promising that weight.

Use selected Dolma and Nemotron partitions first, inspect overlap before fetching every source, and measure the effect of language/type filtering. Do not assume discarding “15% multilingual” leaves a known usable count. The Marin data guide describes the source audit, global dedup and source revision requirements.

A finite work queue for the all-in case

Reserve **576 GPU-days** for all pre-hero GPU work; the remaining 2,304 GPU-days fund 24 hero days. CPU data engineering and storage have their own budget. Six whole-cluster GPU-days is a capacity equivalent, not proof that the dependency chain fits six calendar days. Source access, CPU cleanup and a usable integration corpus should precede the timed GPU window; otherwise shorten/rescope the hero run rather than assuming an unfinished corpus will arrive on time. If GPU embedding/annotation shares these 96 GPUs, it must fit the same preparation cap. Data access work should begin before the rental clock if possible, but that is a schedule choice, not a free-resource assumption.

Stage	Proposed GPU-day ceiling	Deliverable and stop condition
Environment, trainer, export/eval parity and systems sweep	72	Stable baseline, exact token accounting, restart and measured throughput
Reference HP screening and neighborhood refinement	32	Interior stable LR/batch/decay neighborhood at short and longer horizons
Data-mixture screen	16	Proportional baseline plus seven targeted variants, two seeds; prune to budget
Size/horizon transfer ladder	160	At least three widths and independent horizon variation; measured HP plateaus and fit
Fresh long-horizon and depth/hero-shape holdouts	160	At least one 200–250 TPP check, transfer bracket, realistic full-topology rehearsal
Standard-control comparison / GPU data annotation	64	Must be priced before starting; use measured annotation throughput
Contingency and final acceptance evaluation	72	Expand boundary searches or resolve failures; do not spend twice
Total	**576**	Hard cap; revise the scope before exceeding it

These are **allocation ceilings**, not derived claims that all desirable runs fit. Before submission, estimate each job as $\mathrm{GPUdays}_j = D_j f_j/(r_j q_j 86400)$ from its actual shape and measured rate. Sum the manifest, including aborted/failed runs and shared-prefix branches. The earlier 1,024-GPU-day experiment matrix does not fit this preparation cap and is not reused wholesale.

The mixture-screen manifest gives explicit phase weights for seven recipes and a retained-inventory rule for the proportional baseline; regenerate it with `python scripts/industry_mixture_screen.py`. Proposed interventions transfer mass from a named donor bucket, so weights remain normalized. They are hypotheses, not extracted Marin optima.

A concrete 80-run queue is generated by `python scripts/industry_run_matrix.py`: nine reference LR/batch combinations plus three decay refinements at 10B tokens, three reference LR brackets at 100B, eight data recipes × two seeds × 20B, a width-only 256/512/768 ladder at 100/200/300 TPP × three LR brackets (27 runs), eighteen token-batch brackets at the three widths and 100/300 TPP using the fitted LR law, three fresh width-1024/depth-12 holdout runs at 220 TPP, and a 5B-token hero-shaped rehearsal. The initial width family has depth 8, fixed 88 experts/top-8 and fixed query/KV head ratio. It tests width transfer; the fresh holdout also changes depth.

The run ledger projects about 325.2 GPU-days of standalone training at the optimistic 250-TFLOP rate, plus 72 engineering and 72 contingency, leaving about 106.8 GPU-days within the cap for slower small jobs, additional batch/decay checks, a standard-control family and GPU data annotation. Small jobs can be much slower than the hero; replace projected rates

after the first benchmarks. The queue is a reviewable experiment design: absolute LR centers, feasible token-batch rounding, control geometry, reduced data pools and the trainer configuration are still to be resolved, and no jobs have been submitted. The 5B-token rehearsal does not substitute for a long stability qualification.

If the cap cannot fund representative long-horizon holdouts plus the new loop implementation, prioritize a standard MoE and a well-validated data/optimizer recipe. Recurrence remains a candidate hypothesis. Do not sacrifice correctness and target-regime validation to retain every architecture experiment.

Work gates and responsibilities

1. **Source/pilot gate:** source owners deliver revision/access status, retained-token estimates and dedup/decontamination samples. Systems owners deliver a basic trainer and an evaluator on the same tokenizer. The first milestone is a ~10B-token integration release, not a multi-terabyte download without a verified loader.
2. **Capacity gate:** inventory U_b against the table, measure annotated-data throughput and temporary disk, and confirm that the proposed horizon fits the GPU budget. Select a provisional stored size and document serving memory/latency targets.
3. **Data gate:** compare a capped proportional mix against targeted quality/code/math/low-web variants. Use fixed target eval weights and proxy pools with matching bucket repetition. Freeze the winning release before the expensive HP fits.
4. **Transfer gate:** tune jointly near the proposed LR/batch/decay, vary size and horizon, and validate predictions on fresh larger/longer runs. Width-only transfer and depth-changing transfer need separate evidence. Do not transfer dense AdamW coefficients to recurrent MoE without testing.
5. **Architecture gate:** compare tuned standard and looped MoE at matched runtime and target data. Require correct repeated-layer gradients, stable routing and cached decoding. If the recurrent model does not earn its overhead, choose the standard control.
6. **Launch gate:** an immutable corpus, resolved trainer config/container, measured end-to-end throughput, compatible inference/export path, resume test, fixed evaluation protocol and remaining GPU ledger are all present. No calibrated loss forecast is claimed unless we actually performed and validated the optional loss-law study.

What is complete versus still an experiment

Complete in this repository: papers and available LaTeX/code collected; official trainer trees downloaded and pinned; Marin's public experiment records inspected; analytical budget and bucket exposures generated; the science/engineering guides rendered with equations and decision charts.

Still to do on data/compute infrastructure: obtain selected source access, download and prepare the retained corpus, implement/qualify a selected trainer configuration and custom loop if retained, run the bounded experiments, and evaluate actual checkpoints. Neither prepared trillions of tokens nor an empirically chosen hero architecture is claimed.